

# **EP 391 BalloonSat Design Project**

## **Easy-Solution Inc.**

*Enrique Amaya, Tyler, Wharton & Misael Jimenez*

*EP391 – Microcomputers & Electronic Instrumentation*

*Department of Physical Sciences | Embry-Riddle Aeronautical University*

*Instructor: Dr. Mubarak Mujawar*

*April 2026*

### **Objective:**

The objective of this project was to design and implement a multi-functional sun tracking system capable of monitoring environmental and system level parameters while tracking a light source. The system integrates four primary subsystems: a tracking subsystem, a temperature sensing subsystem, a power monitoring subsystem, and a telemetry subsystem. The tracking subsystem uses photodiode-based sensors and a stepper motor to orient toward a light source, while the temperature subsystem measures environmental conditions using thermistors and signal conditioning circuits. The power monitoring subsystem measures the voltage of power supply batteries and the magnitude of current of power draw to ensure safe and efficient operation. All sensor data is processed by a microcontroller and transmitted via UART to a MATLAB-based ground station, where the data is displayed and analyzed in real time. This integrated approach demonstrates the application of analog signal conditioning, embedded systems, and data acquisition in a unified engineering design.

## Power Source Subsystem:

For this first subsystem within this BalloonSat project, the principal focus of creating this system was to provide a stable and self-contained electrical power source for all major payload electronics. Due to a variety of reasons such as having analog circuitry, and that the microcontroller has a voltage limit of 5V, the DC power from the batteries had to be changed through a positive and negative voltage regulator. This setup provided to be of great help since by setting these two power regulators, the system was able to derive and create a balanced power environment for the other subsystems that needed both positive and negative rails, such being the case for many of the operational amplifiers used. Therefore, with this in mind, the primary objective for this subsystem was to supply 5V to both BalloonSat amplifiers, and supply at least 7V to the stepper motor.

For the power subsystem, the components that were used to make it fully operational included:

- Three 9V batteries
- LM317: Adjustable Positive Voltage Regulator
- LM337: Adjustable Negative Voltage Regulator
- 0.1  $\mu F$  Tantalum capacitors for both regulators
- 1  $\mu F$  Tantalum capacitors for both regulators
- 330 $\Omega$  resistor for LM317
- 150 $\Omega$  resistor for LM337

The battery subsystem was designed around the principle of converting an unregulated DC input into more stable regulated outputs. It can be explained that, using the lab manual 5 from the EP391 course, voltage regulators are used to convert an unsteady DC source

into a lower but steadier output voltage, with the tradeoff that some energy is dissipated as heat in the regulation process. The LM317 was selected as the positive voltage regulator because it provides an adjustable regulated positive output and is well suited for generating a stable supply rail. Similarly, the LM337 was selected as the negative voltage regulator because it provides an adjustable negative output, making it appropriate for creating the complementary negative rail required by the analog circuitry. Together, these two regulators allowed the subsystem to generate controlled positive and negative supply voltages rather than relying directly on raw battery output.

The capacitors were included to improve voltage stability and reduce unwanted fluctuations in the regulator outputs. The 1  $\mu F$  and 0.1  $\mu F$  tantalum capacitors were chosen because the regulator circuits require external filtering elements to improve steadiness and transient response. The fixed resistors and 10 k $\Omega$  potentiometers were used to set and adjust the regulator output voltages, allowing the final supply rails to be tuned to the desired values during testing. Schematics for the voltage regulators equations and the H-bridge used to setup and power the motor can be found in Appendix E.

Furthermore, a third 9V battery acting as a DC power source was used to power the motor used for the tracking system, where it is connected to the ground and onto pin 8 from the H-bridge used for that subsystem. The schematics for the H-bridge can be seen in appendix E, from figures 4, 5 and 6. It is also briefly explained in the tracking subsystem section of this report.

## Power Monitoring:

The objective of the power monitoring subsystem is to track the voltage levels of the batteries used to power the satellite, as well as the rate of power draw from the batteries. Our power monitoring subsystem only tracks one battery's voltage due to a limited number of pins on the microprocessor, and it was not able to successfully track the power draw. The voltage monitoring system consists of two inverting amplifiers. The first amplifier uses two  $100\text{K}\Omega$  resistors, one  $200\text{K}\Omega$  resistor, and a TL082 operational amplifier. The second amplifier uses two  $39\text{K}\Omega$  resistors and a TL082 operational amplifier. The positive terminal of a battery is sent into the first amplifier, which has a gain of  $-1/3$ . The schematic for these amplifiers can be seen in Appendix E figure 3.

The output of this amplifier is then sent into the other amplifier with a gain of  $-1$ , resulting in a net gain of  $1/3$ . The output of the second amplifier is then sent to the microprocessor to be read. The first inverting amplifier is used to reduce the magnitude of the voltage being sent into the microprocessor, as it can only receive up to  $5\text{V}$  while the batteries can go up to  $9\text{V}$ . A gain of less than one is needed, and only inverting amplifiers can accomplish this. The second amplifier is needed to turn the negative output of the first into a positive voltage, as the microprocessor cannot receive negative voltages.

The power draw monitoring system is intended to monitor the current being pulled from the battery. This is more difficult to accomplish because current cannot be measured in parallel with a system. Our initial plan to measure the current was to use a shunt resistor in series with the battery, which would be placed before the voltage regulator. Dividing the voltage drop across the resistor by the resistance of the resistor

would then provide the current flowing across the resistor.

A problem in this approach arose when trying to send this voltage to the microprocessor to be read. There was only one analog pin remaining on the microprocessor, while two pins were required to get the voltage drop across the shunt resistor (voltage before and after the resistor). An alternative was to use a voltage divider alongside a non-inverting amplifier to get a current reading, but this setup drew a significant portion of the positive power supply's voltage and thus was not used. The schematic can be seen in figure 8 in Appendix E.

## Tracking Subsystem:

The objective of the tracking subsystem is to determine the angular displacement of some light source (in a real-world application this would be the sun, for the demo it is a flashlight) from an initial position. The tracking subsystem consists of three photodiodes, three transimpedance amplifiers, three  $330\Omega$  resistors, a stepper motor, an H-bridge, and a custom 3D printed part to fix the photodiodes to the motor shaft. Each transimpedance amplifier consists of a TL082 operational amplifier and a  $10\text{M}\Omega$  resistor. The circuit schematic for the amplifiers is shown in Figure 1. Note that the stepper motor receives power from a different battery than the rest of the satellite, as it requires more power to generate torque. The circuit schematic for the stepper motor is shown in Figure 2. The photodiodes were placed  $0.625$  inches from the center of the shaft and were spaced  $50$  degrees apart. This meant that the tracking subsystem could only "see" the front side of the system, and if the light source is behind the system then it will not be able to see it.

To ensure that the satellite doesn't get stuck looking at nothing, if there is no light detected then the motor will continue rotating in the last known direction of the light source until light is detected. The logic which the tracking system used to determine if it was looking directly at the light source is shown in figure 9. One of the limitations of the tracking system's design was the torque generated by the stepper motor. If the payload was too heavy, the motor would not be able to spin. Additionally, if there was something resisting the motion of the motor (such as wires pulling back on the photodiodes), then the motor would not be able to spin. To overcome these limitations, the 3D part used to hold the photodiodes must be lightweight and must not have a large radius in order to avoid creating a large moment arm. Additionally, careful cable management is required to ensure that the wires connected to the photodiodes have enough slack as to not pull back on the motor when it spins.

The tracking subsystem needed to be calibrated to have an accuracy of at least ten degrees. In order to accomplish this, trial runs were done where the light source was stationary, and the tracking system ran until it came to rest, i.e. when it decided that the light source was straight in front of it. Depending on how accurate it was when it came to rest, the threshold to consider the light source to be in the middle of the sensor system was increased or decreased. If the threshold is too low, then the tracking system will not be accurate enough. If the threshold is too high, then the tracking system may ever recognize that the light source is head-on. We decided that the middle photosensor's reading should be 15x that of the side photosensors. This threshold is quite large because the middle photosensor is a green LED, while the side photosensors are white LEDs, which means the middle photosensor will have a lower

reading at the same incoming light intensity. If all photosensors were white, then the threshold would likely be something like 1.5x.

## **Telemetry Subsystem**

Telemetry: The telemetry subsystem was calibrated using a combination of analytical and empirical methods. Voltage measurements were calibrated analytically using the known resistor ratio of the voltage divider, allowing ADC counts to be scaled directly to system voltage. Current was modeled using a linear calibration based on sensor output voltage, with offset and sensitivity constants applied. Temperature measurements were calibrated using the thermistor B-parameter equation, converting ADC readings to resistance, and then to temperature, with additional empirical offsets applied to improve accuracy. The sun angle was determined using an open loop step count method based on the stepper motor's known angular resolution, providing a theoretical estimate of orientation without feedback correction.

## **Temperature Subsystem:**

For the thermistor subsystem within this BalloonSat circuit, the main objective consisted in being able to determine a way to record the temperatures at two separate locations on the payload, as well as transmitting those values into a running, real-time telemetry subsystem. The idea behind this was essentially to establish both a method to measure the temperature behavior of this satellite payload and to correctly identify any fluctuations in the temperature readings being collected. The core principle behind designing this subsystem was to

satisfy the required operating interval between  $-20^{\circ}\text{C}$  to  $60^{\circ}\text{C}$  degrees, while at the same time providing a signal that could be reliably acquired by the ATmega644PA microcontroller on its analog-to-digital converter. To accomplish this, the final design combined thermistor-based voltage dividers, non-inverting amplifier stages for analog conditioning, and embedded software that converted ADC measurements into temperature using the thermistor B-equation.

Among its components, the thermistor subsystem included:

- 2 LM05-103 K NTC thermistors
  - Nominal resistance of  $10k\Omega$  at  $25^{\circ}\text{C}$
  - Resistance tolerance of  $\pm 10\%$
  - B constant of 4038 K
  - Operating temperature range of  $-55^{\circ}\text{C}$  to  $150^{\circ}\text{C}$
- 1 analog interface circuit for each sensor
- ATmega644PA microcontroller
- ADC channels for digitization
- UART transmission for telemetry reporting
- TL082 dual operational amplifiers for both thermistors

The two thermistors were intended to be mounted near opposite ends of the payload structure so that the system could measure both local temperature and temperature differential across the BalloonSat body. In the implemented software, the two thermistor outputs were assigned to ADC3 and ADC4 (found in Appendix E figure 4). The final code used a 5 V ADC reference, a 10-bit ADC resolution, a fixed series resistor of  $5230\ \Omega$ , and an analog gain of 1.038 for the amplifier stage. It is also important to note that the

TL082 op-amp was used for both thermistors in a non-inverting amplifier setup.

Both NTC thermistors proved to be well suited due to their temperature ranges and tolerances, allowing for the measurement of local temperatures and temperature differentials across the BalloonSat payload. The following figure represents a Multisim schematic of the thermistor structure, which was taken as reference from Lab13 from the EP391 lab manual. While the Multisim design captures the thermistor interface circuit, it does not represent the complete implemented subsystem. Key differences lie in that both VCC+ and VEE- have a 5V input, as well as the microcontroller not being included in this schematic. The purpose of these schematic models is to demonstrate the analog front-end used to generate and condition the temperature-dependent voltage, whereas the actual hardware system also includes the ATmega644PA microcontroller, ADC sampling on channels ADC3 and ADC4, and software-based conversion from ADC counts to temperature.

The thermistor subsystem was designed around the principle that an NTC thermistor changes resistance as temperature changes. Since the AVR microcontroller cannot measure resistance directly, each thermistor had to be incorporated into a circuit that converted resistance variation into a measurable voltage. To achieve this, each thermistor was placed in a 5 V voltage-divider network. In this configuration, the thermistor formed one half of the divider, and a fixed resistor formed the other half, so the divider output voltage changed as the thermistor resistance changed with temperature. This divider output served as the analog signal representing the measured thermal condition.

## **Result Analysis & Conclusion:**

During our trial run we observed a steady voltage of roughly 6V from a battery. We saw how the angle of the light source changed over time, and we also observed a temperature of roughly 23 °C on either side of the satellite. We were not able to measure the current drawn from the battery. All results were successfully displayed via MATLAB on a wireless ground-station.

Some possible improvements for the satellite include a stronger motor which could better overcome resistance from the wires in the photosensor system pulling back on the motor. Alternatively, having the entire device rotate instead of just the photosensor system would prevent wires from being pulled, though this is only feasible for an airborne device (which a real BalloonSat would be).

Another solution to this problem would be a method for the photosensor system to wirelessly transmit data to the microprocessor, thus eliminating the need for wires on the photodiodes. The cheapest solution, however, would be to have longer cable and more precise cable management. Another area of improvement would be the photosensor system itself. Due to a limitation of available materials, one of the photodiodes used was a green LED, which meant that it was less sensitive to light than the two other clear LEDs. Using a clear LED would allow more light to reach the photosensor, and thus would allow the satellite to detect fainter light. Additionally, using more LEDs would allow for an even more sensitive photosensor system. The LEDs also seem to be more recipient to light at certain angles, so doing some experimentation to figure out the best angle to place the LEDs at could improve the photosensor system's sensitivity to light.

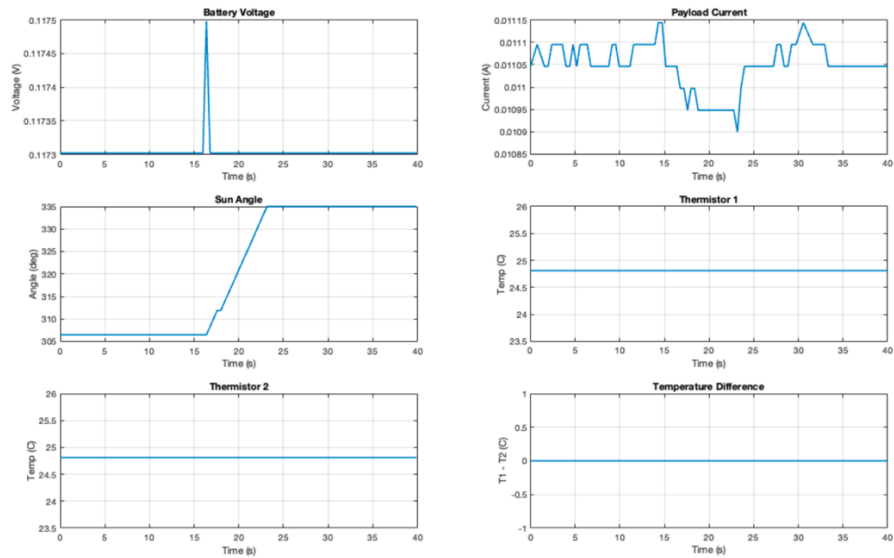
The most important improvement would be to have more analog pins on the microprocessor, as this would be necessary to implement the monitoring subsystem to all the power supply batteries. The easiest way this could be achieved is by using a microprocessor with more analog pins. A more advantageous solution would be to use multiple microprocessors at the same time.

This is particularly beneficial because having multiple microprocessors in parallel would allow multiple readings to be taken simultaneously, as a single microprocessor can only read one analog pin at a time. It would also allow a microprocessor to be placed on the motor along with the photosensor system, which would remove much of the resistance in the motor's motion. This solution would be more difficult to implement because all microprocessors would need to wirelessly transmit data to the same ground station. Overall, the BalloonSat design could see significant improvements by adding just a few extra components.

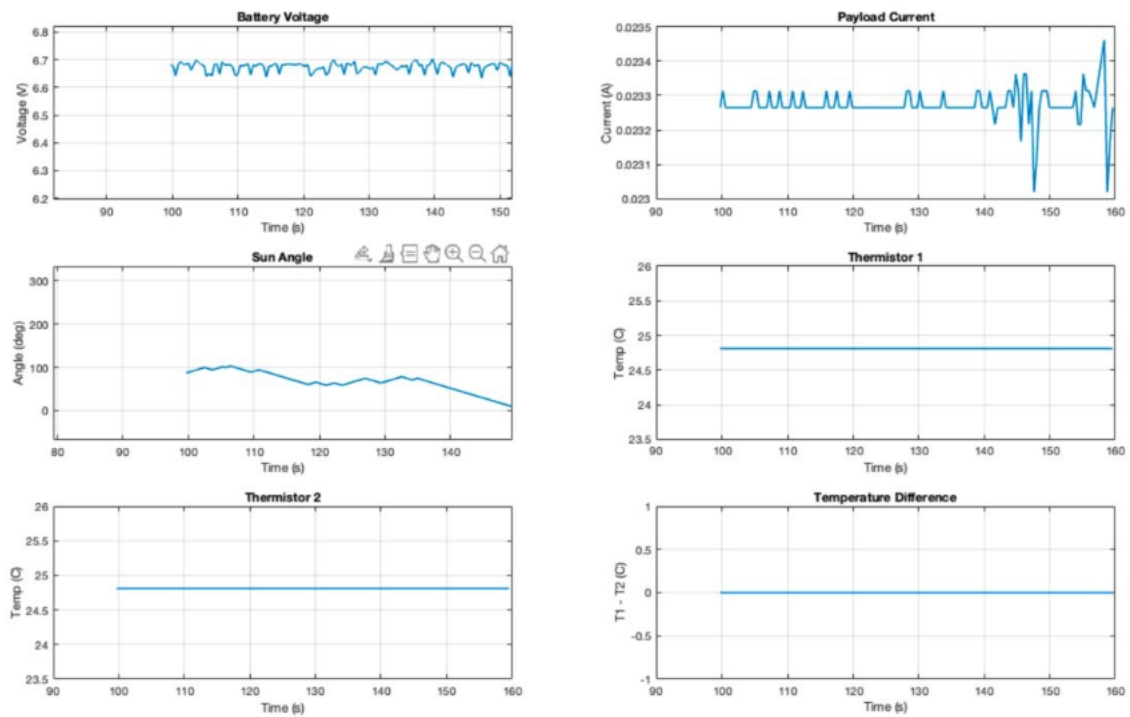
## Appendix A

Data:

Testing Data:



Actual results from 5 minutes run:



## Appendix B

*Equations:*

Telemetry:

- Converts counts into voltage:

$$V_{adc} = ADC * \frac{5}{1023}$$

Temperature Subsystem:

- Voltage Gain for Non-Inverting Amplifier:  $A_v = 1 + \frac{R_f}{R_{in}}$
- Microcontroller voltage measured from ADC:  $V_{amp} = \frac{(ADC\ Counts)(V_{ref})}{1023}$
- Original voltage divider voltage:  $V_{div} = \frac{V_{amp}}{G}$
- Thermistor Resistance:  $R_{th} = R_{series} \frac{V_{div}}{V_{ref} - V_{div}}$
- Thermistor Temperature:  $T = \frac{1}{\frac{1}{T_0} + \frac{1}{B} \ln\left(\frac{R_{th}}{R_0}\right)}$



## Appendix C

Motor Code:

```
#define F_CPU 1000000UL

#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include <avr/interrupt.h>
#include <stdlib.h>
#include <math.h>

#define UART_UBBR_VALUE 12UL //Set baud rate = 9600 bps at double speed (U2X=1)

//Not sure if timer stuff is needed

//Moves motor based on three readings going to ADC0, ADC1, and ADC2
//Photosensor (LED) connected to amplifier provides light reading
//Use amplifier C w/ clear LED and Rf of at least 10M0hms

//If none of the diode readings meet some threshold, spin the satellite in the last
movement direction until light source is found
//If left diode reading is largest, move clockwise
//If middle diode reading is largest, stay still
//If right diode reading is largest, move counterclockwise
//Maybe make it so left or right reading must be at least 5% larger than middle? or
something like that - may reduce error to noise

//Motor takes 202 steps for one full rotation
//Angle per step is 360/202 = 1.782178218
//Print to two decimal places

//CURRENT PROGRESS
//Motor correctly moves according to three light readings

//TO DO
//Determine the angle of the motor
//Provide some means of calibration? Motor angle reading will likely become inaccurate
over time

void uart_init();
void motorMove(int direction);
void write_uart(char c[]);

int step[4]={0x01,0x04,0x02,0x08}; //Stepping Sequence in Hex;
int i=0; // Initialize variable i
```

```

int lightLeft;
int lightMiddle;
int lightRight;
int lightMin = 30; //Minimum light reading to accept as actual source, anything less is
counted as noise
int readCount = 0;
int dir = 0;

float angle = 0;
float anglePerStep = 1.782178218;
int angleInt;
int angleDec;

int main()
{
    uart_init();
    TCCR1B |= (1<<WGM12); // CTC (Clear Timer on Compare) with OCR1A value
    OCR1A = 65535; // Timer1 should be compared to ~67 seconds
    sei(); // Enable global interrupts

    DDRB |= 0x0F;

    TIMSK1 |= (1<<OCIE1A); // Output compare interrupt
    TCCR1B |= (1<<CS10) | (1<<CS12); // CLK/1024 in other words f(effective) = f/1024
    while(1){}
}

void uart_init()
{
    //Initializes uart
    //Starting at ADC0
    /*Set Baud rate*/
    ADMUX |= (1<<REFS0);
    ADMUX |= (0<<ADLAR);
    UBRR0H = UART_UBBR_VALUE >> 8;
    UBRR0L = UART_UBBR_VALUE;
    /*Frame format: 8 data bits, no parity, 1 stop bit*/
    UCSR0C = (0<<USBS0) | (1<<UCSZ01) | (1<<UCSZ00);
    /*Enable Transmit and Receive*/
    UCSR0B = (1<<RXEN0) | (1<<TXEN0);
    /*Enable Double Speed (U2X=1) for 9600 bps*/
    UCSR0A = (1<<U2X0);
    ADCSRA |= (1<<ADEN); // Enable ADC
    ADCSRA |= (1<<ADSC); // Auto trigger enable
    ADCSRA |= (1<<ADIF); //Enable interrupts
    ADCSRA |= (1<<ADSC); // Start ADC

```

```

}

void write_uart(char c[])
{
//writes messages to uart, mainly for debugging
while(*c != '\0')
{
while((UCSR0A & (1<<UDRE0)) == 0); // Wait for Transmit Buffer to Empty
UDR0 = *c; // Transmit the character
c++; // Increment the pointer to point to the next character
}
}

void motorMove(int direction) {
//moves motor in a certain direction (CW = 0, CCW = 1)
PORTB &= 0xF0; // Turn off the stepper motor only
_delay_ms(50); // Delay the time between turning off and on the motor
PORTB |= step[i]; // Cause the stepper motor to make a step
_delay_ms(50); // Delay the time between the step and shutting off the motor
if (direction) {
i = (i+1)%4; //Modulus is so stepping sequence will repeat
angle = angle + anglePerStep;
}
else {
i = (i+3)%4; //This is like doing i=((i-1)+4)%4. The minus one is to reverse step
direction, then the plus four keeps it as a positive number.
angle = angle - anglePerStep;
}
angleInt = angle;
angleDec = abs((angle-angleInt)*100);
}

ISR(ADC_vect) // When each ADC completes, this routine is executed
//does something based on IR sensor reading
{
ADCSRA &= ~(1<<ADEN); // Disable ADC

//Read each LED
if (readCount == 0)
{
lightLeft = ADCW;
//Set ADMUX to read ADC1 (MUX = 01)
ADMUX |= (1<<MUX0);
ADMUX &= ~(1<<MUX1);
}
else if (readCount == 1)

```

```

{
lightMiddle = ADCW;
//Set ADMUX to read ADC2 (MUX = 10)
ADMUX &= ~(1<<MUX0);
ADMUX |= (1<<MUX1);
}
else if (readCount == 2)
{
lightRight = ADCW;
//Set ADMUX to read ADC0 (MUX = 00)
ADMUX &= ~(1<<MUX0);
ADMUX &= ~(1<<MUX1);
}

readCount++;

//After each LED has been read, do logic
if (readCount >= 3)
{
readCount = 0;
char buffer[80];

if ((lightLeft < lightMin) && (lightMiddle < lightMin) && (lightRight < lightMin))
{
//Not pointed anywhere near light, move in last direction until light is sensed.
motorMove(dir);

sprintf(buffer, "No light. --- Left: %d, Middle: %d, Right: %d, Angle: %d.%d\n\r",
lightLeft, lightMiddle, lightRight, angleInt, angleDec);
write_uart(buffer);
}
else if ((lightLeft > lightMiddle * 0.8) && (lightLeft > lightRight))
{
//Light source is to the left, move counterclockwise
dir = 1;
motorMove(dir);

sprintf(buffer, "Left, moving counterclockwise. --- Left: %d, Middle: %d, Right:
%d, Angle: %d.%d\n\r", lightLeft, lightMiddle, lightRight, angleInt, angleDec);
write_uart(buffer);

}
else if ((lightRight > lightMiddle * 0.8) && (lightRight > lightLeft))
{
//Light source is to the right, move clockwise
dir = 0;
motorMove(dir);
}
}

```

```

sprintf(buffer, "Right, going clockwise. --- Left: %d, Middle: %d, Right: %d,
Angle: %d.%d\n\r", lightLeft, lightMiddle, lightRight, angleInt, angleDec);
write_uart(buffer);
}
else {
//Light is in middle, dont move
sprintf(buffer, "Middle, staying still. --- Left: %d, Middle: %d, Right: %d,
Angle: %d.%d\n\r", lightLeft, lightMiddle, lightRight, angleInt, angleDec);
write_uart(buffer);
}
_delay_ms(100); //Wait some time to start sensing again
}
_delay_ms(50); //some extra delay
TCNT1=0; // Reset the timer back to zero
ADCSRA |= (1<<ADEN); // Enable ADC
ADCSRA |= (1<<ADSC); // Start ADC
}

ISR(TIMER1_COMPA_vect) // this interrupt is fired when timer 1 equals OCR1A
//does something when timer overflows, may not be neccessary
{
write_uart("Timer reset.\n\r"); // Write the string to the UART
}

```

Thermistor code:

```
/* Temperature Subsystem Test Code

Still deciding if fit everything into one code or not

*/

// ?????????????????????????????????????????????????????????????????
// Temperature Subsystem – BalloonSat EP391 Final Project
// Reads two thermistors on ADC3 and ADC4, converts to Celsius
// using the B-equation from Lab 13, transmits over UART
// ?????????????????????????????????????????????????????????????????

#define F_CPU 1000000UL // Tell the AVR library our clock is 1MHz
// Must be defined BEFORE including delay.h

#include <avr/io.h> // AVR register names (ADMUX, ADCSRA, etc.)
#include <util/delay.h> // _delay_ms() function
#include <stdio.h> // sprintf() for formatting strings
#include <math.h> // log() for the B-equation

// UART
#define UART_UBBR_VALUE 12UL // Baud rate register value for 9600bps
// at 1MHz with U2X double-speed enabled

// ADC
#define VREF 5.0 // ADC reference voltage = AVcc = 5V
#define ADC_MAX 1023.0 // 10-bit ADC max count (2^10 - 1)

// THERMISTOR CONSTANTS
#define R0 10000.0 // NTCLE100E3103HB0 resistance at 25C (10k?)
#define T0_K 298.15 // Reference temperature in Kelvin (25C = 298.15K)
#define BETA 3977.0 // B-coefficient from NTCLE100E3103HB0 datasheet

// CIRCUIT CONSTANTS
#define R_SERIES 5230.0 // R3 series resistor in voltage divider (5.23k?)
#define AMP_GAIN 1.038 // Non-inverting op-amp gain = 1 + (Rf/Rin)
// = 1 + (200k? / 5230k?) = 1.038

// TEMPERATURE OFFSET
#define TEMP_OFFSET_1 -2f
#define TEMP_OFFSET_2 -2f

// FUNCTION PROTOTYPES
```

```

void uart_init(void);
void uart_write_char(char c);
void uart_write_string(char *s);
void adc_init(void);
uint16_t adc_read(uint8_t channel);
float adc_to_tempC(uint16_t adc_count);

// MAIN
int main(void)
{
    char buffer[120]; // String buffer for UART output
    uint16_t adc3, adc4; // Raw ADC counts from each thermistor
    float t1, t2; // Calculated temperatures in Celsius

    uart_init(); // Set up UART so we can print over serial
    adc_init(); // Set up ADC hardware

    while (1) // Loop forever, reading and transmitting continuously
    {
        // Read raw 10-bit ADC values (0-1023) from each thermistor channel
        adc3 = adc_read(3); // Thermistor 1 on ADC3 (pin PA3)
        adc4 = adc_read(4); // Thermistor 2 on ADC4 (pin PA4)

        // Convert raw ADC counts to Celsius using B-equation
        t1 = adc_to_tempC(adc3) + TEMP_OFFSET_1;
        t2 = adc_to_tempC(adc4) + TEMP_OFFSET_2;

        // Split floats into integer and decimal parts for AVR sprintf
        int t1_int = (int)t1;
        int t1_dec = (int)((t1 - t1_int) * 10);
        if (t1_dec < 0) t1_dec = -t1_dec;
        int t2_int = (int)t2;
        int t2_dec = (int)((t2 - t2_int) * 10);
        if (t2_dec < 0) t2_dec = -t2_dec;

        sprintf(buffer, "T1=%d.%d C, T2=%d.%d C, ADC3=%u, ADC4=%u \r\n",
            t1_int, t1_dec, t2_int, t2_dec, adc3, adc4);
        uart_write_string(buffer);

        _delay_ms(500); // Wait 500ms before next reading (2 reads/sec)
    }
}

// ?????????????????????????????????????????????????????????????
// ADC TO TEMPERATURE CONVERSION (B-equation from Lab 13)
// Input: raw 10-bit ADC count (0-1023)

```

```

// Output: temperature in Celsius
// ?????????????????????????????????????????????????????????????
float adc_to_tempC(uint16_t adc_count)
{
float v_amp, v_div, r_th, temp_K;

// Step 1: Convert ADC count to voltage at the ADC pin
// Formula: V = (count / 1023) * VREF
// e.g. count=512 ? V = (512/1023)*5.0 = 2.502V
v_amp = ((float)adc_count * VREF) / ADC_MAX;

// Step 2: Undo the op-amp gain to recover the raw divider voltage
// The op-amp amplified the divider voltage by AMP_GAIN (1.038)
// so we divide back out to get what the divider actually produced
v_div = v_amp / AMP_GAIN;

// Safety clamps: prevent divide-by-zero in Step 3
// if v_div is 0 or 5V exactly, the resistance formula breaks
if (v_div <= 0.001f) v_div = 0.001f;
if (v_div >= VREF - 0.001f) v_div = VREF - 0.001f;

// Step 3: Convert divider voltage to thermistor resistance
// Circuit: +5V ? R_SERIES ? (v_div) ? Rtherm ? GND
// By voltage divider: v_div = 5V * Rtherm / (R_SERIES + Rtherm)
// Solving for Rtherm: Rtherm = R_SERIES * v_div / (VREF - v_div)
r_th = R_SERIES * (v_div / (VREF - v_div));

// Step 4: B-equation (from Lab 13) to get temperature in Kelvin
// From Lab 13: 1/T = 1/T0 + (1/B) * ln(Rt/R0)
// Solved for T: T = 1 / (1/T0 + (1/B)*ln(Rt/R0))
temp_K = 1.0f / ((1.0f / T0_K) + (1.0f / BETA) * log(r_th / R0));

// Convert Kelvin to Celsius and return
return temp_K - 273.15f;
}

// ?????????????????????????????????????????????????????????????
// UART INITIALIZATION
// Sets up serial communication at 9600 baud, 8N1
// Same configuration as the tracking subsystem code
// ?????????????????????????????????????????????????????????????
void uart_init(void)
{
// Set baud rate by splitting the 12 value across high and low registers
UBRR0H = UART_UBBR_VALUE >> 8; // Upper 8 bits (will be 0)
UBRR0L = UART_UBBR_VALUE; // Lower 8 bits = 12

UCSR0A = (1 << U2X0); // Enable double speed mode for accuracy

```



```

UCSR0B = (1 << TXEN0);           // Enable transmitter only (no RX needed)
UCSR0C = (1 << UCSZ01) | (1 << UCSZ00); // 8-bit data frame
}

// Send a single character over UART
void uart_write_char(char c)
{
    // Wait until the transmit buffer is empty before writing
    while (!(UCSR0A & (1 << UDRE0)));
    UDR0 = c; // Load character into transmit register
}

// Send a full string by looping through each character
void uart_write_string(char *s)
{
    // Keep sending characters until null terminator '\0' is reached
    while (*s) { uart_write_char(*s++); }
}

// ?????????????????????????????????????????????????????????????????
// ADC INITIALIZATION
// Sets reference voltage and prescaler, enables ADC
// ???????????????????????????????????????????????????????????????
void adc_init(void)
{
    // REFS0 = 1 means use AVcc (5V) as the ADC reference voltage
    ADMUX = (1 << REFS0);

    // ADEN = enable the ADC hardware
    // ADPS2 + ADPS1 = prescaler of 64 ? ADC clock = 1MHz/64 = 15.6kHz
    // ADC needs 50-200kHz clock; 15.6kHz is within that range
    ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1);
}

// ???????????????????????????????????????????????????????????????
// ADC READ
// Selects a channel, triggers a conversion, waits, returns result
// Input: channel number (0-7)
// Output: 10-bit ADC result (0-1023)
// ???????????????????????????????????????????????????????????????
uint16_t adc_read(uint8_t channel)
{
    // Set the channel in the lower 4 bits of ADMUX
    // 0xF0 mask preserves the reference voltage bits we set in adc_init()
    ADMUX = (ADMUX & 0xF0) | (channel & 0x0F);

    ADCSRA |= (1 << ADSC);           // Start the ADC conversion
    while (ADCSRA & (1 << ADSC));    // Wait until conversion is complete
}

```

```
// ADSC bit clears automatically when done
return ADC;           // Return the 10-bit result
}
```

Telemetry code:

```
Telemetry code: clear;
clc;
close all;

%Serial port info
portName = "/dev/cu.usbserial-D309PKGS";
baudRate = 9600;
timeoutSec = 3;
plotWindowSec = 60;

% Calibration constants

% ADC
VREF = 5.0;
ADC_MAX = 1023.0;

% Voltage divider for voltage channel
% Vin = Vadc * ((R_top + R_bottom)/R_bottom)
R_TOP_VOLTAGE = 10000.0;
R_BOTTOM_VOLTAGE = 10000.0;

% Current conversion
% Example: I = (Vadc - offset) * sensitivity
CURRENT_ZERO_OFFSET = 0.0;
CURRENT_SENSITIVITY = 1.0;

% Thermistor constants from your temperature code
R0 = 10000.0;
T0_K = 298.15;
BETA = 3977.0;
R_SERIES = 5230.0;
AMP_GAIN = 1.038;
TEMP_OFFSET_1 = -2.0;
TEMP_OFFSET_2 = -2.0;

%Arrays
time = [];
voltage = [];
current = [];
sun_angle = [];
temp1 = [];
temp2 = [];
tempDiff = [];

%Serial setup
```

```

fprintf('Telemetry Ground Station\n\n');
try
    s = serialport(portName, baudRate);
    configureTerminator(s, "LF");
    s.Timeout = timeoutSec;
    flush(s);
    fprintf('Connected to %s at %d baud.\n\n', portName, baudRate);
catch ME
    fprintf('Could not open serial port.\n');
    fprintf('%s\n', ME.message);
    return;
end

%Window/Figure
fig = figure('Name','Telemetry Ground Station','NumberTitle','off', ...
    'Color','w', 'Position',[100 100 1200 700]);

%Timer
tic;
fprintf('Elapsed Time: 00:00');
nbytes = length('Elapsed Time: 00:00');

%Main loop
while ishandle(fig)

    % Timer
    t = toc;
    minutes = floor(mod(t,3600)/60);
    seconds = floor(mod(t,60));
    timeStr = sprintf('%02d:%02d', minutes, seconds);

    fprintf(repmat('\b', 1, nbytes));
    nbytes = fprintf('Elapsed Time: %s', timeStr);

    gotValidData = false;

    try
        line = readline(s);

        if isstring(line) || ischar(line)
            line = strtrim(string(line));

            if strlen(line) > 0
                data = split(line, ",");

                % Expected: adcVoltage,adcCurrent,angle,adcTherm1,adcTherm2
                if numel(data) == 5
                    vals = str2double(data);

                    if all(~isnan(vals))
                        gotValidData = true;

                        adcV = vals(1);

```

```

        adcI = vals(2);
        A_deg = vals(3); % already-computed angle from AVR
        adcT1 = vals(4);
        adcT2 = vals(5);

        %Voltage conversion
        Vadc = (adcV * VREF) / ADC_MAX;
        V = Vadc * ((R_TOP_VOLTAGE + R_BOTTOM_VOLTAGE) /
R_BOTTOM_VOLTAGE);

        %Current conversion
        Iadc = (adcI * VREF) / ADC_MAX;
        I = (Iadc - CURRENT_ZERO_OFFSET) * CURRENT_SENSITIVITY;

        %Thermistor 1 conversion
        T1 = adcCountToTempC(adcT1, VREF, ADC_MAX, AMP_GAIN, R_SERIES,
R0, T0_K, BETA) + TEMP_OFFSET_1;

        %Thermistor 2 conversion
        T2 = adcCountToTempC(adcT2, VREF, ADC_MAX, AMP_GAIN, R_SERIES,
R0, T0_K, BETA) + TEMP_OFFSET_2;

        Td = T1 - T2;

        % Store
        time(end+1) = t;
        voltage(end+1) = V;
        current(end+1) = I;
        sun_angle(end+1) = A_deg;
        temp1(end+1) = T1;
        temp2(end+1) = T2;
        tempDiff(end+1) = Td;
    end
end
end
end

catch
    % keep looping
end

% Rolling window
if ~isempty(time)
    idx = time >= max(0, t - plotWindowSec);
else
    idx = [];
end

subplot(3,2,1); cla;
if ~isempty(time), plot(time(idx), voltage(idx), 'LineWidth',1.5); end
title('Battery Voltage'); xlabel('Time (s)'); ylabel('Voltage (V)'); grid on;

subplot(3,2,2); cla;
if ~isempty(time), plot(time(idx), current(idx), 'LineWidth',1.5); end
title('Payload Current'); xlabel('Time (s)'); ylabel('Current (A)'); grid on;

```

```

subplot(3,2,3); cla;
if ~isempty(time), plot(time(idx), sun_angle(idx), 'LineWidth',1.5); end
title('Sun Angle'); xlabel('Time (s)'); ylabel('Angle (deg)'); grid on;

subplot(3,2,4); cla;
if ~isempty(time), plot(time(idx), temp1(idx), 'LineWidth',1.5); end
title('Thermistor 1'); xlabel('Time (s)'); ylabel('Temp (C)'); grid on;

subplot(3,2,5); cla;
if ~isempty(time), plot(time(idx), temp2(idx), 'LineWidth',1.5); end
title('Thermistor 2'); xlabel('Time (s)'); ylabel('Temp (C)'); grid on;

subplot(3,2,6); cla;
if ~isempty(time)
    plot(time(idx), tempDiff(idx), 'LineWidth',1.5);
    title('Temperature Difference');
    xlabel('Time (s)');
    ylabel('T1 - T2 (C)');
    grid on;
else
    axis off;
    if gotValidData
        text(0.05,0.7,'Receiving telemetry...','FontSize',12);
    else
        text(0.05,0.7,'Waiting for valid UART data...','FontSize',12);
    end
end
end

drawnow limitrate;
pause(0.25);
end

clear s;
fprintf('\n');

%Local Function
function tempC = adcCountToTempC(adc_count, VREF, ADC_MAX, AMP_GAIN, R_SERIES, R0,
T0_K, BETA)
    v_amp = (adc_count * VREF) / ADC_MAX;
    v_div = v_amp / AMP_GAIN;

    if v_div <= 0.001
        v_div = 0.001;
    end
    if v_div >= VREF - 0.001
        v_div = VREF - 0.001;
    end

    r_th = R_SERIES * (v_div / (VREF - v_div));
    temp_K = 1.0 / ((1.0 / T0_K) + (1.0 / BETA) * log(r_th / R0));
    tempC = temp_K - 273.15;
end

```

## Appendix E:

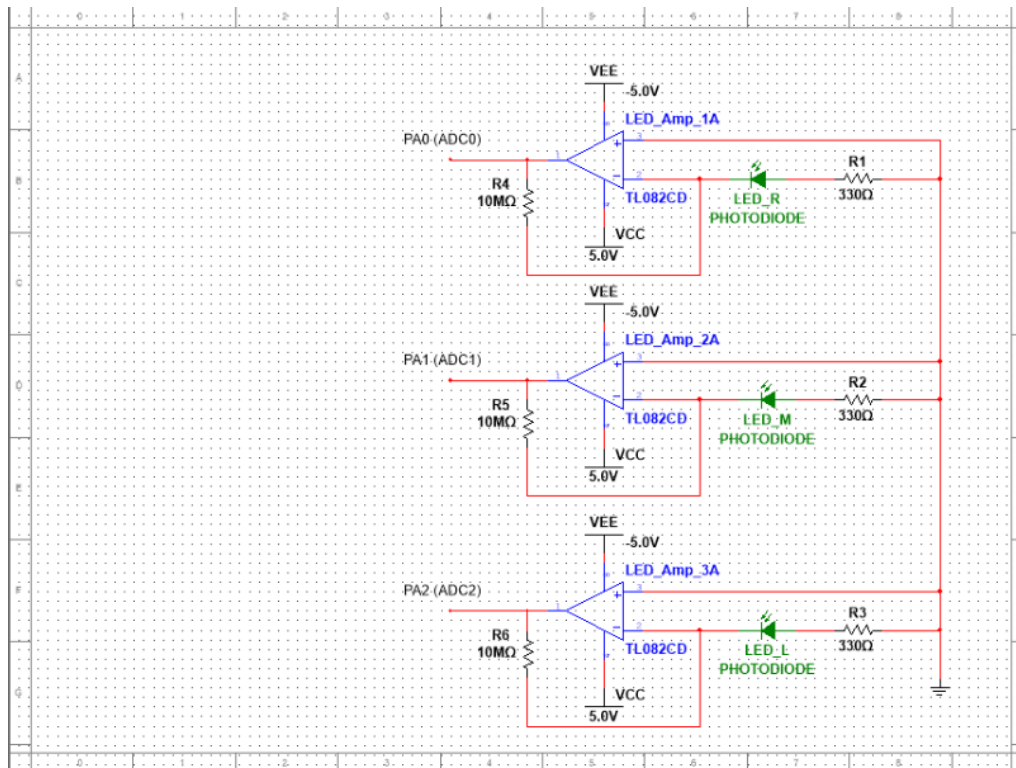


Figure 1: LED Transimpedance Amplifier

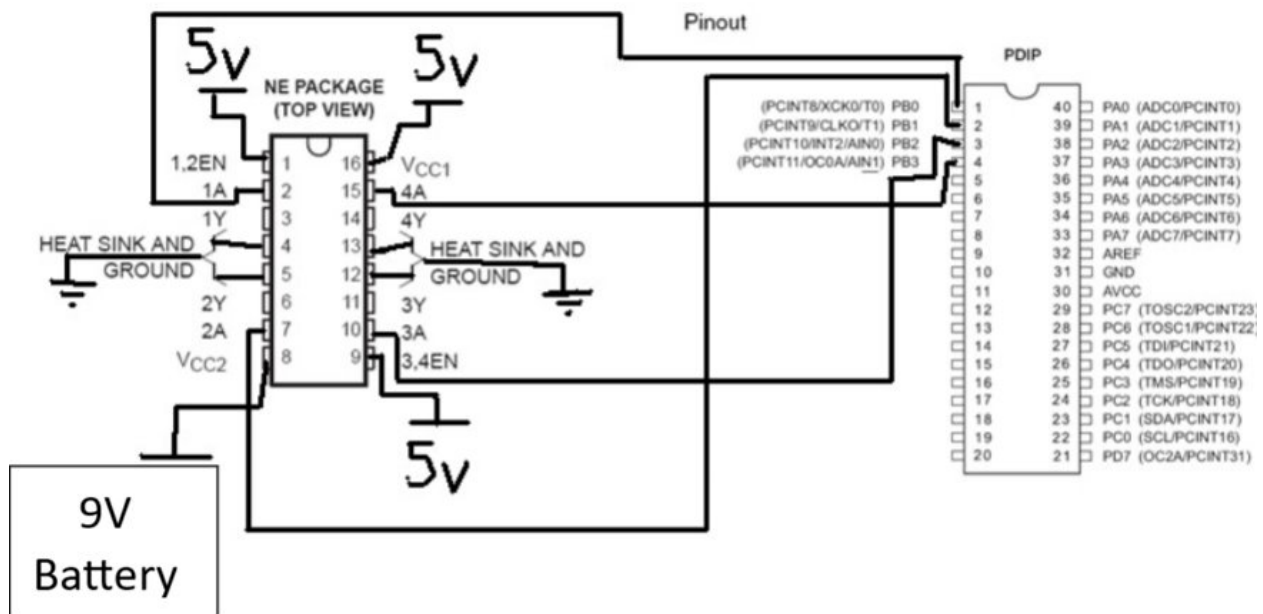


Figure 2: Stepper Motor & H-Bridge

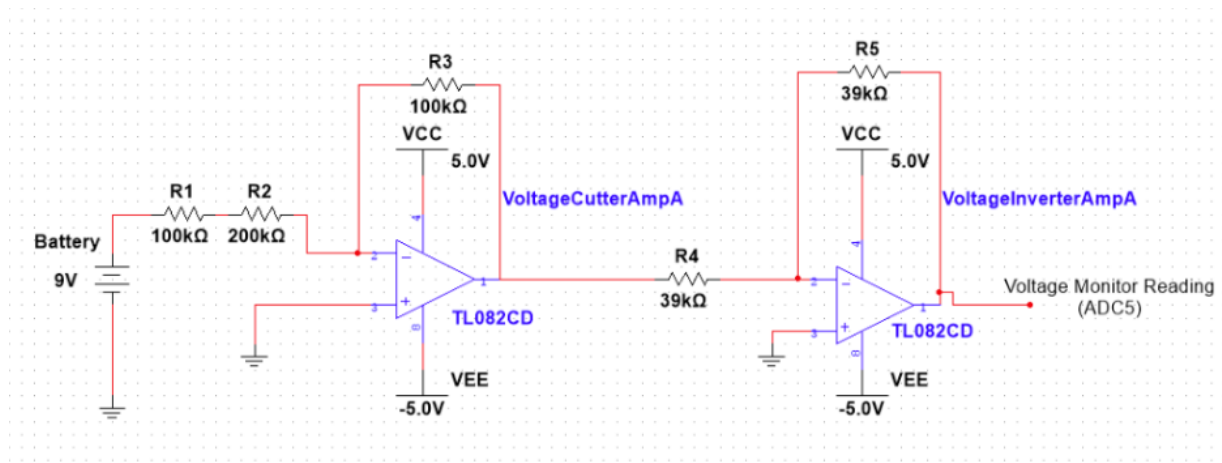


Figure 3: Voltage Monitor system consisting of two inverting amplifiers with a total gain of  $1/3$

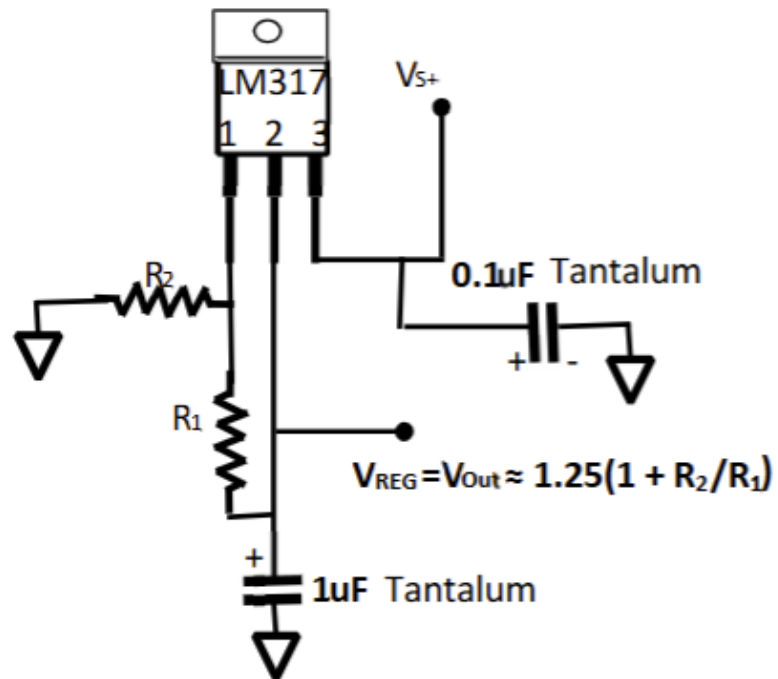


Figure 4: Supplement Positive Voltage Regulator

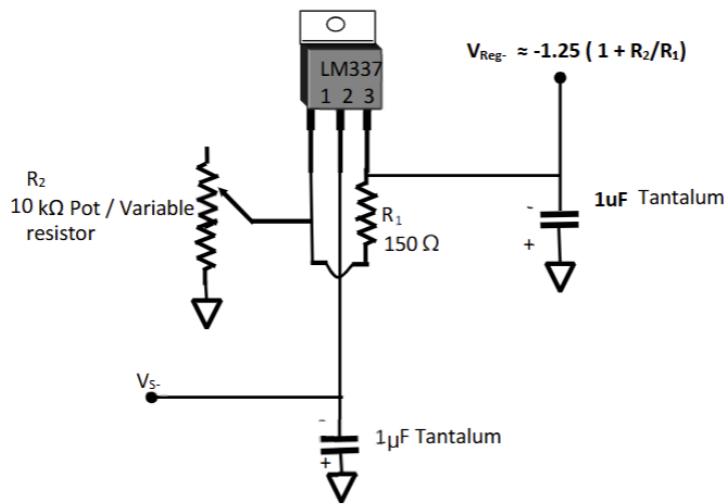


Figure 5: Supplement Negative Voltage Regulator

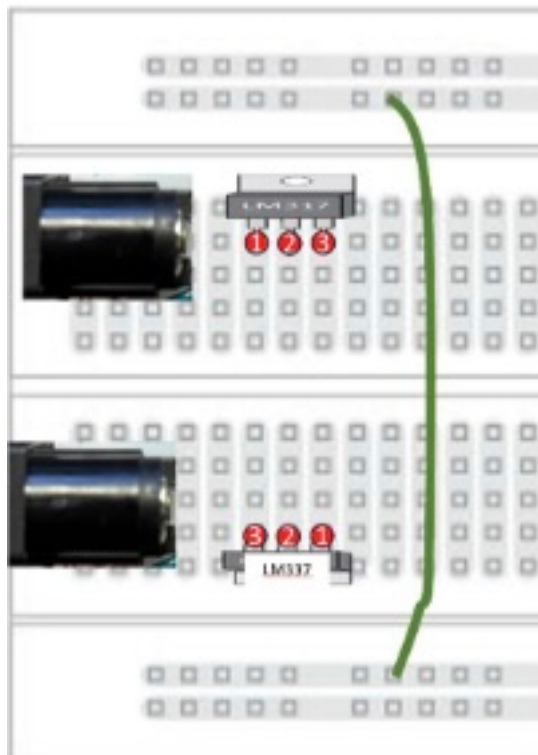


Figure 6: Breadboard setup for both positive and negative voltage regulators with AC power.



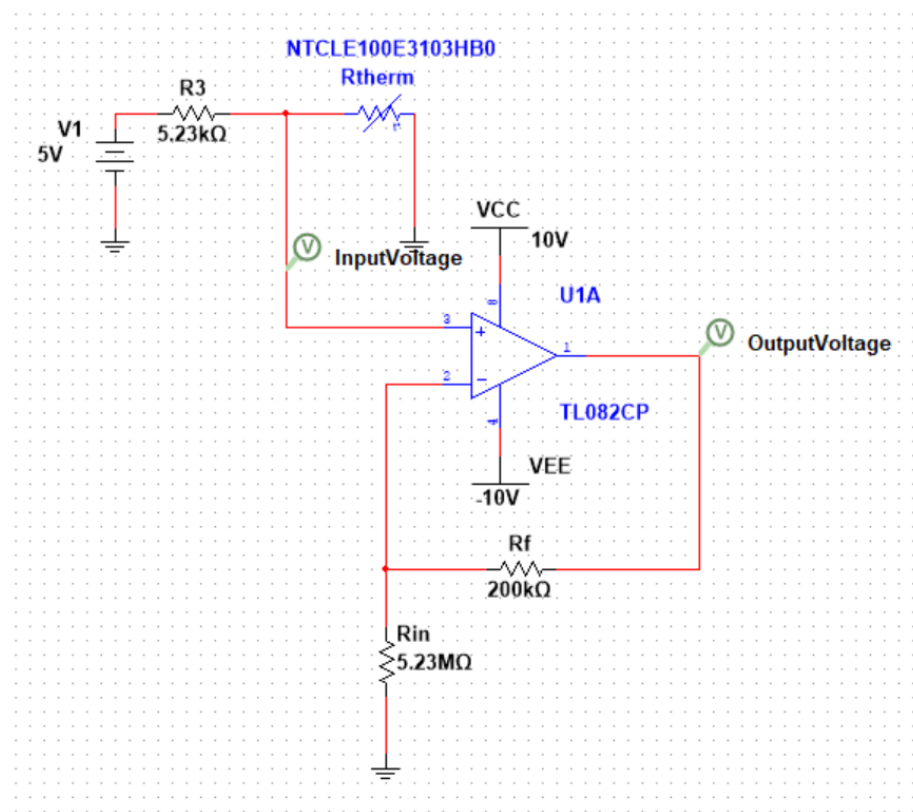


Figure 7: Non-Inverting Amplifier setup for thermistor measuring ranges from  $-20^{\circ}\text{C}$  to  $60^{\circ}\text{C}$

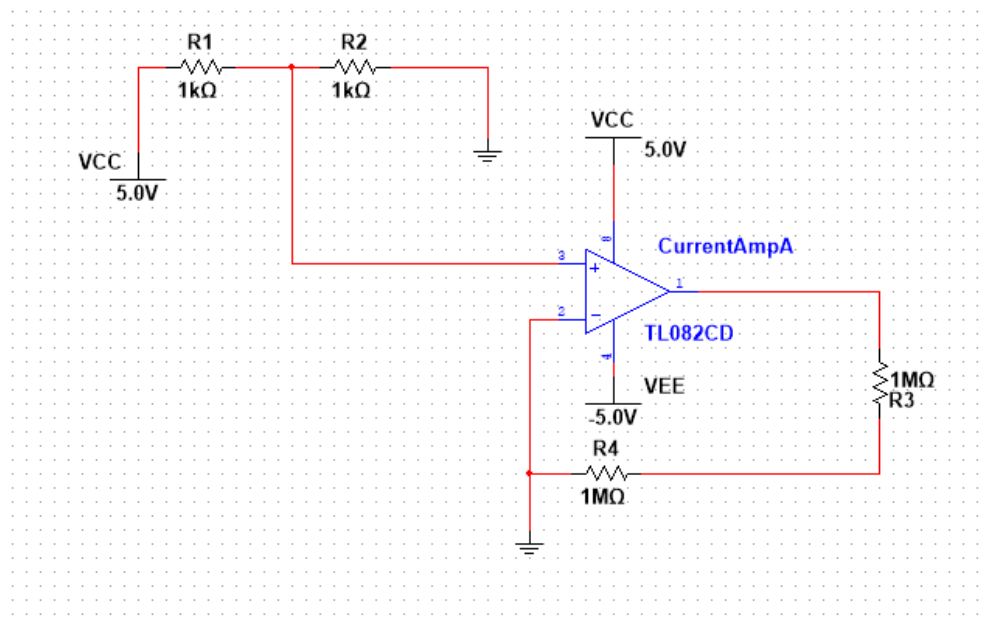


Figure 8: Voltage Divider attempt to create a current monitor

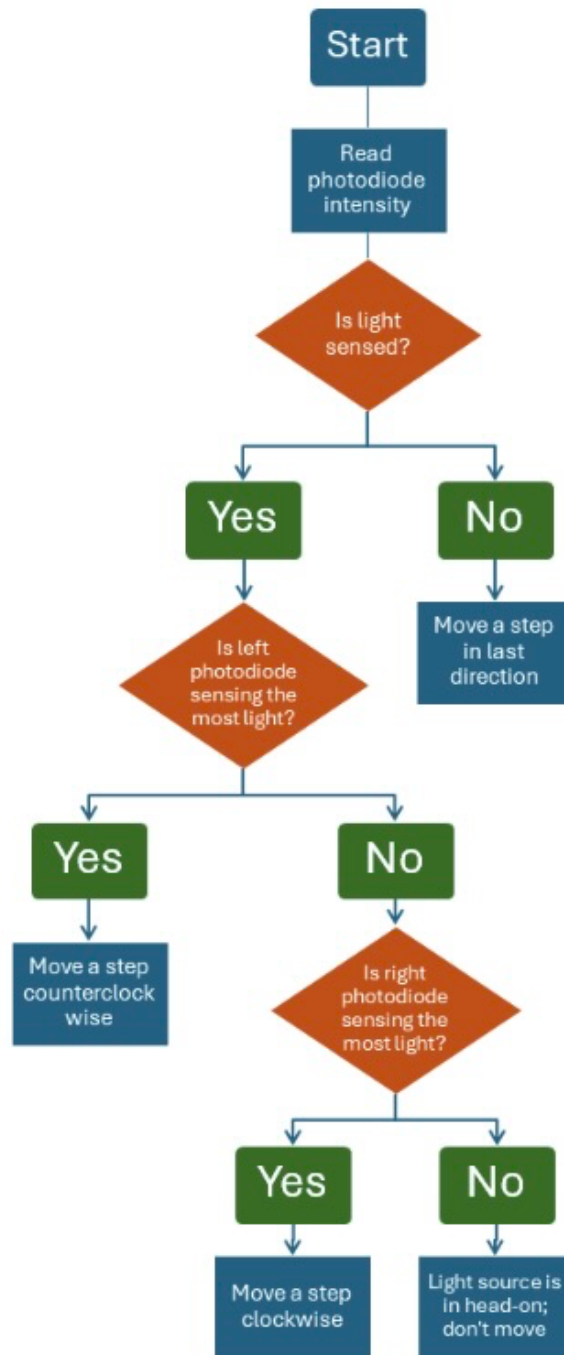


Figure 9: Track Subsystem Flowchart